

A I 时代的编程指南

# 上手工具



Charles Tao 著，与 Claude 协作完成

[books.stickmancharles.com](https://books.stickmancharles.com) · v1.0

正文 CC BY-NC-ND 4.0 · 代码 MIT

署名: Charles Tao, 《AI 时代的编程指南》, [books.stickmancharles.com](https://books.stickmancharles.com)

# AI 时代的编程指南 · 第二册《上手工具》

第一册给了你地图，这一册把手放到方向盘上。

作者：Charles Tao（与 Claude 协作完成） 版本：v1.0 · 2026 年 7 月 · 系列共五册，本册难度  
★★

## 小序

这一册只练我们真实的工作方式。我们的主战场有两个：VS Code 里的 Claude 插件（贴身搭档，坐在你旁边一起改）和 Claude Code 网页版（云端分身，派活出去、收货回来）。终端是配角——我们很少亲自开它，但它是 AI 的手脚，所以第 4 章要把它彻底看懂。

开读之前，先做一件事：建一块试验田。新建一个叫 `playground` 的文件夹，本册所有"试一试"都在里面做。它存在的意义就是被随便折腾、随便搞砸——在试验田里砸过的坑，正式项目里就不会再掉进去。

界面细节（按钮长相、菜单位置）会随版本更新，以你眼前的为准；但本册教的工作循环不会变。

## 目录

1. 你的驾驶舱：把 VS Code 认全
2. 贴身搭档：VS Code 里的 Claude 插件工作流
3. 云端分身：Claude Code 网页版怎么用
4. 命令行生存包：组合技与肌肉记忆
5. Git 核心循环：工作区、暂存区与四步操作
6. GitHub 实战：建仓、上传、修改一个仓库
7. 分支与 PR：正式的协作流程
8. 时光倒流：四种深度的撤销
9. 冲突不可怕：亲手拆一次"炸弹"
10. 认识一个仓库：拿到新项目先看什么

11. 给 AI 立规矩: CLAUDE.md 与技能

12. 综合演练: 从空文件夹到上线网址

附: 本册新词

---

# 第 1 章 你的驾驶舱：把 VS Code 认全

VS Code 是我们每天坐进去的驾驶舱，值得花十分钟把每块仪表认全。

最左边一竖条图标叫活动栏，五个常用的：资源管理器（文件树，项目的目录结构）；搜索（全项目范围的搜索和替换，比一个文件一个文件翻快一百倍）；源代码管理（重点！它就是 Git 的图形界面，下文细说）；运行调试；扩展（插件商店，Claude 插件就装在这里）。中间是编辑区：每个打开的文件一个标签页，拖动标签可以左右分屏对照着看。底部一条状态栏，左下角那个小小的名字千万别忽略——那是你当前所在的 Git 分支，第 7 章讲分支时你会庆幸自己早就知道它在哪。

两个"任意门"快捷键，值得练成肌肉记忆：`Cmd/Ctrl + P`，输入文件名片段直接跳到任何文件；`Cmd/Ctrl + Shift + P` 打开命令面板——把你想对编辑器做的任何事打几个字母出来，相当于"编辑器自己的终端"。再按 `Ctrl + `` 会在底部弹出集成终端：它就是第一册第 7 章那个终端，只是嵌进了编辑器——AI 敲命令，多数时候就敲在这里，当着你的面。

重点看源代码管理面板。你改动过的文件会自动列在这里；点开任何一个，看到的是差异视图（Diff）：红色是删掉的，绿色是新增的。文件旁的 `+` 号、上方输入框旁的 `√`、菜单里的"推送"——这些按钮背后分别就是 `git add`、`git commit`、`git push`（第 5 章细讲）。图形界面和命令行是同一件事的两张脸，两张脸都认识，你才能既自己点得动，又看得懂 AI 在终端里做的是同一回事。

## 关键领悟

VS Code 里几乎每个按钮背后，都是一条你在第一册见过名字的命令。

## 试一试

随便改动一个文件里的一个字，去源代码管理面板看它出现，点开看红绿差异，再用文件上的"放弃更改"图标把它撤销掉。

## 第 2 章 贴身搭档：VS Code 里的 Claude 插件工作流

插件形态的 Claude 像坐在你旁边的搭档：它看得到你的项目，改动直接落在你的文件上。正因为它离你的真实代码这么近，工作循环比什么都重要。六步，每一步都有存在的理由：

第一步，把需求说清。用第一册第 13 章的骨架（背景、目标、约束、例子），并且尽量指名道姓：引用具体文件、选中具体代码块再提问，永远好过凭空描述。"改一下登录逻辑"和"改 `login.ts` 里 `handleSubmit` 这个函数"是两个世界。第二步，先要计划。"先告诉我你打算改哪些文件、怎么改，别动手。"看过计划再放行。第三步，逐文件审差异。它改完后，用第 1 章学的差异视图一个文件一个文件看：先扫文件清单——它动的范围和计划一致吗？有没有碰不该碰的东西？（红线思维，时刻在线）——再看红绿内容本身。第四步，接受或打回。不满意就具体地说哪里不对，别勉强接受再自己补救。第五步，跑起来验证。按第一册第 14 章的金字塔：能跑、功能对、边界情况。第六步，`commit` 存档。验证通过立刻存档，给下一轮实验垫好回滚点。

再加两条会话纪律。其一：一个会话只干一件事。修 bug 的会话别顺手聊新功能——上下文混了，两件事都容易走样。其二：新任务开新会话。会话太长，早先的约定会被挤出 AI 的视野（第一册第 12 章讲过它的"遗忘"）；新会话等于给它一张干净的桌子。

### 关键领悟

插件的价值不是"它改得快"，是"你审得住"。六步循环里，属于你的的是第一、三、五步——说清、审查、验证，这三步恰好是 AI 替代不了的。

### 试一试

下一个任务，强制自己走完整六步——尤其别跳过"先要计划"。数一数它的计划和你的想象差了几处。

## 第 3 章 云端分身：Claude Code 网页版怎么用

如果说插件是坐在身边的搭档，网页版就是派驻云端的身分。心智模型三句话：你把 GitHub 仓库授权给它；它在云上的一间独立工作室（沙箱）里克隆代码、按你的任务书干活；干完把成果打包成一条分支和一个 PR 递回来（这两个词第 7 章正式讲，先记住：那是一份“请过目”的改动包）。

它和插件是分工，不是竞争。适合派给身分的活：任务大而明确、可以写成一份完整任务书的（“把所有页面的旧版按钮统一换成新组件”）；你人不在电脑前、想用手机或浏览器发起的；想同时并行推进好几件互不相干的事的。不适合的活：需要你本机环境或本机密钥才能跑的；边聊边想、方向还没定的模糊探索——那种活适合贴身搭档。

用它，你只有三个动作，每个都在考验你的基本功。派活：因为它在云上独自干活、你没法随时插话，任务书的质量几乎决定一切——背景、目标、约束、例子，一样都别省，红线更要写明。这是异步协作的第一法则：规格写多清楚，成果就有多接近。收货：去 GitHub 打开它提的 PR，在 Files changed 里逐文件审红绿差异——和第 2 章第三步一模一样，只是换了个地方。裁决：满意就合并，不满意就在 PR 里写下具体的修改意见，让它再来一轮。

### 关键领悟

网页版是你“提需求”能力的试金石——没有追问和补救的机会，一次说清的功力全在这里现形。

### 试一试

挑一个小而明确的任务派给网页版，全程只通过任务书和 PR 交流，不用插件救场。收货时给自己的任务书打个分。

## 第 4 章 命令行生存包：组合技与肌肉记忆

第一册第 7 章给了你十个命令的"单词表"，这一章教"造句"。目标依旧不变：自己偶尔用得上，更重要的是看懂 AI 的每一步。

先练四个肌肉记忆，它们不解决问题，但让你在终端里不狼狈：**Tab** 补全——路径和文件名打头几个字母就按 Tab，终端替你补全，这是终端世界最重要的一个键，从此告别拼写错误；**↑** 方向键——翻出刚才敲过的命令，改一改再跑；**Ctrl + C**——强行中断当前正在跑的程序（卡住了、跑错了，都是它）；**clear**——清屏，还你一张干净桌子。

再学三个组合技。通配符 **\*** 是"任意内容"：**ls \*.md** 列出所有 Markdown 文件。**grep -r "关键词"**，在当前目录下递归搜索每一个文件——想知道某个函数在项目里哪些地方被用过，一行搞定（这正是 VS Code 搜索面板背后的原理）。看日志专用组合：**tail -20 文件** 看文件最后二十行，**tail -f 文件** 则持续追着看，新日志一来就滚出来——盯着服务的实时输出时就是它。

还有两个日常小工具：**which 命令名** 和 **命令名 --version**，查某个工具装没装、装的什么版本（AI 排查环境问题时最先跑的就是这类命令）；**open .**（Mac）或 **explorer .**（Windows）把当前目录在图形界面里打开——两个世界随时互通。系统差异说一句就够：本书命令以 Mac / Linux 为准，Windows 的 PowerShell 大多相通，遇到不认的命令，把报错丢给 AI。

### 关键领悟

终端的高手气质不来自记住多少命令，来自 Tab、↑、Ctrl+C 这三个键用得毫不犹豫。

### 试一试

在 **playground** 里用 **grep -r** 搜一个词，再用 **tail** 看任意文件的最后几行。全程强制自己用 Tab 补全每一个路径。

## 第 5 章 Git 核心循环：工作区、暂存区与四步操作

第一册讲了 Git 是时间机器，这一章补上当时略过的一块关键拼图——三个区域。搞懂它，Git 的一切命令瞬间通顺。

你的文件其实同时活在三个地方。工作区：你正在编辑的这些文件本身，桌面上摊开的稿纸。暂存区（Staging Area）：一张打包台——`git add` 的意思就是"把这份改动放上打包台"。仓库：存档柜——`git commit` 把打包台上的所有东西封成一箱，贴上一句话标签，放进柜子。远端还有第四个地方：远程仓库（GitHub 上那份），`git push` 把新封的箱子运上云。

为什么中间要多一个打包台？因为它让你挑着打包。你可能同时改了五个文件，其中三个是修 bug、两个是顺手调样式——分两次 `add`、两次 `commit`，历史就清清楚楚："修了登录 bug"、"调整了按钮样式"。一次 `commit` 只装一件事，将来翻档案、回滚某一步时，你会感谢今天的自己。

于是核心循环就是四步：`git status`（我在哪、什么改了、什么已在打包台——迷路先 `status`，它是指南针）→ `git add` 文件（上台）→ `git commit -m "一句话说清这次做了什么"`（封箱）→ `git push`（运上云）。配两个观察窗：`git diff` 看还没上台的改动，`git log --oneline` 看存档编年史。回到第 1 章：源代码管理面板里的改动列表、`+` 号、`√`、"同步"，对应的就是这四步——AI 在终端里跑的，也是这四步。

### 关键领悟

Git 的几十个命令，九成日子里只用这四步加两个观察窗。剩下的，等出状况再学（第 8、9 章）。

### 试一试

在 `playground` 里改两个文件，故意只 `add` 其中一个，`commit`，再 `add` 另一个，再 `commit`——用 `git log --oneline` 看看"挑着打包"出来的两条干净历史。

## 第 6 章 GitHub 实战：建仓、上传、修改一个仓库

这一章回答一个最实际的问题：手上有本地项目，怎么把它变成 GitHub 上的一个仓库？以及之后怎么持续修改它。

第一步，云上建一个空仓。GitHub 网页右上角 New repository：起名（小写加连字符是惯例）、选 Public（公开，全世界可读——秘密纪律立刻拉满）还是 Private（私有，受邀才可见）。关键一条：不要勾选自动生成 README 之类的初始化选项——让云上是个完全的空仓，否则它有一份历史、你本地另有一份历史，第一次推送就会打架。

第二步，三行咒语，把本地推上去。建完空仓，GitHub 会把这三行摆在你面前：

`bash`

```
git remote add origin https://github.com/你的用户名/仓库名.git
git branch -M main
git push -u origin main
```

逐行翻译，咒语立刻祛魅。第一行：给云上那个仓库在本地存个“联系人”，名字叫 `origin`（惯例名，就像手机里存“家”）。第二行：把当前主分支改名为 `main`（现行惯例的主干名）。第三行：把 `main` 推给 `origin`，`-u` 的意思是“记住这对关系”——从此以后每天只需要裸敲 `git push`。首次推送会要求登录 GitHub 授权，跟着提示走即可。刷新网页，看到自己的代码出现在云上——记住这一刻的感觉。

之后的修改，就是第 5 章的循环：改 → add → commit → push，网页上随之更新。两个补充场景：反方向——云上已有仓库想搬到新电脑，`git clone 地址` 整仓复制（第一册讲过）；网页直改——README 里改个错别字，不必回本地，网页上点铅笔图标改完直接 commit。但注意：网页上改过之后，本地就旧了，回到本地先 `git pull` 把云上的新箱子拉下来。由此得出一条终生习惯：动手之前先 pull，让本地永远从最新状态出发。

关键领悟

`remote add`、`branch -M`、`push -u` 这三行一辈子只在建仓时用；真正天天用的，还是那四步循环加一个 pull。

试一试

把 `playground` 推上 GitHub（走完整流程，别让 AI 代劳）。然后在网页上用铅笔改一个字，回本地 `git pull`，观察改动流回来。

---

## 第 7 章 分支与 PR：正式的协作流程

第一册说过分支是"平行宇宙"。这一章把完整流程走一遍——它同时也是你和 Claude Code 网页版协作的标准流程。

开宇宙：`git switch -c feat-search`（新建并切换到名叫 `feat-search` 的分支；旧写法 `git checkout -b` 同义，AI 两种都会用）。此刻左下角状态栏的分支名变了——你已站在平行宇宙里，怎么折腾都不影响 `main`。在宇宙里干活：正常的改、`add`、`commit`。把宇宙推上云：`git push -u origin feat-search`。发起合并申请：刷新 GitHub，页面顶端会出现一条黄色提示"Compare & pull request"，点它，写清这个分支做了什么，创建 PR（Pull Request）——一份正式的"请过目"。审查：PR 页面的 `Files changed` 标签页就是全部改动的红绿差异，可以逐行留言。裁决：满意点 `Merge`，宇宙并入主线；随后删掉这条已完成使命的分支（GitHub 会顺手给你删除按钮）。回家：本地切回 `main`（`git switch main`），`git pull` 把合并成果拉下来。

一个人开发，为什么也要走这套？两个理由。其一，PR 是强制的"过目"环节——直接推 `main`，改动就无声无息进了主干；走 PR，你必须把全部差异正眼看一遍，这对审查 AI 的产出尤其珍贵。其二，这就是收货流程：第 3 章说过，Claude Code 网页版交活的方式就是一条分支加一个 PR——你在本章练的每一步，都是在练怎么收它的货。分支起名有个通行小惯例：`feat-` 开头是新功能，`fix-` 开头是修复，一眼可读。

### 关键领悟

`main` 是正式出版物，分支是草稿纸，PR 是编辑审稿。让一切改动先当草稿，被看过，再出版。

### 试一试

下次让 AI（插件或网页版皆可）改任何东西时，要求它"开分支、提 PR"，然后你在 GitHub 网页上完成一次正式审查与合并。

## 第 8 章 时光倒流：四种深度的撤销

搞砸是日常，Git 的价值就在于每种深度的"砸"都有对应的退路。按错误走了多远，分四层——层数越深，动作越要慎重。

第一层：改了，还没 `add`。最常见也最无风险。`git restore 文件名`，或者干脆在 VS Code 源代码管理面板点那个"放弃更改"图标——改动瞬间蒸发，回到上次存档的样子。第二层：`add` 了，还没 `commit`。东西已在打包台，想拿下来：`git restore --staged 文件名`——只是撤下打包台，改动本身还在工作区，可改可弃。第三层：`commit` 了，还没 `push`。箱子已封，但还没运出去。最简单也最推荐的姿势是不倒带，往前修：直接再改、再 `commit` 一次修正——历史里多一个"修正"存档，诚实又安全。（进阶的"改写上一箱"手法叫 `amend`，规矩只有一条：没 `push` 的才许改写。）第四层：已经 `push` 了。箱子已进公共档案柜。唯一正路是 `git revert` 提交号：Git 自动生成一个反向存档，把那次改动原样抵消——历史上两箱都在，一来一回，清清楚楚。

为什么到了第四层就不能"直接删掉那个 `commit`"？因为那叫改写公共历史（`reset --hard` 加 `force push` 干的就是这事，它们在第一册的红线卡上）：云上的历史别人（包括 AI、包括部署平台）都已经拿走对过账，你单方面篡改，所有人的账本就对不上了。听着耳熟吗——这正是链上合约教过我们的道理：进了公共账本的东西，只能追加修正，不能抹掉重写。Git 在每天用小事故提醒我们这条大原则。

### 关键领悟

口诀一句——没推的可以整理，推了的只能追加。分不清在哪一层时，先 `git status`，再问 AI，别凭感觉敲。

### 试一试

在 `playground` 里把四层各制造一次、各撤销一次。重点体会第四层：`revert` 之后跑 `git log --oneline`，看历史如何"一来一回"。

## 第 9 章 冲突不可怕：亲手拆一次"炸弹"

冲突（Conflict）的发生条件只有一个：两边改了同一处，Git 不敢替你们做主。本地和网页各改了 README 的同一行；你的分支和 main 动了同一个函数；你和 AI 一人一版——都会触发。合并或 pull 时，Git 停下来，把球踢给你。

它的样子第一次见很吓人，看懂了就只是个填空题。冲突的文件里会出现这样的标记：

□

```
<<<<<<< HEAD
你这边的版本
=====
对方那边的版本
>>>>>>> feat-search
```

翻译过来就是 Git 在摊手："上面是你的，下面是他的，你俩自己挑。"拆弹三步：一，做决定——留你的、留对方的，或者动手把两边融成一版；二，删标记——那三行 <<< === >>> 是 Git 的便签，决定完必须删干净；三，收尾——保存、add、commit，冲突解除。VS Code 会把冲突高亮并给出"采用哪边"的快捷选项，点起来更省事——但原始标记必须看得懂，因为 AI 解决冲突时，日志里出现的就是这些原文。

说到让 AI 解冲突：可以，但你要审一件事——两边的意图是否都活了下来。AI 最容易犯的错是为了让红线消失而随手丢掉一边的改动，表面干净，实际丢了功能。审查时别看标记删没删，要看两边原本想做的事是否都还在。

最后是预防，三个习惯让冲突几乎绝迹：动手前先 pull（第 6 章的终生习惯）；小步提交、勤推送，别攒一大包；分支短命——开了尽快合，拖得越久和 main 的分歧越大。

### 关键领悟

冲突不是事故，是 Git 在保护你——它宁可停下来问，也不敢替你猜。拆过一次弹的人，从此不怕这三行标记。

试一试

在 `playground` 里自造一次冲突：网页上改某行，本地改同一行并 `commit`，然后 `git pull`——亲手把它拆掉。

---

## 第 10 章 认识一个仓库：拿到新项目先看什么

打开一个陌生仓库（包括我们自己几个月没碰的老项目），从第一行代码读起是最低效的方式。老手有一套开箱顺序，五分钟建立全局感。

先读 README——项目的说明书：它是什么、怎么跑起来、怎么参与。一个仓库对人友好不友好，README 见分晓。再看 `package.json`——第一册说它是购物清单，这里补上它的另一半身份：快捷按钮面板。里面的 `scripts` 区写着类似 `"dev": "next dev"` 的条目，意思是：敲 `npm run dev`，等于替你敲后面那串长命令。`npm run dev` 的真相就这么朴素——它是项目作者预设的快捷键，`dev` 通常是“本地跑起来”，`build` 是“打包成品”。顺带一条铁律：clone 下来的项目第一件事跑 `npm install`——照购物清单把货搬齐（`node_modules` 不进 Git，得自己装），清单变了也要重跑。

然后扫一眼目录结构：`src` 是源代码本体，`public` 是原样上线的静态文件（图片、图标）。根目录那一排配置文件，认脸即可，不必细读：`tsconfig.json` 是 TS 的规矩（第三册见），`tailwind.config` 是样式的规矩，`vite.config` 或 `next.config` 是构建的规矩，`.github/workflows/` 里藏着自动流水线（第一册说的 CI/CD 本尊），`LICENSE` 声明开源许可。它们像家电说明书——知道在哪，用到再翻。

单独点名一个宝贝：`.env.example`。它是环境变量口袋的清单模板——列出这个项目需要哪些钥匙，但不含钥匙本身（所以它可以进 Git）。拿到新仓库跑不起来，八成是没照着它配出自己的 `.env`。看它，是“这个项目依赖哪些外部服务”的最快答案。

### 关键领悟

读仓库的顺序是 README → package.json → 目录 → .env.example，最后才是代码。全局感先于细节。

### 试一试

按这个顺序把我们的主仓库重新“开箱”一遍，每遇到一个不认识的配置文件，问 AI 一句：“它管什么规矩？”

## 第 11 章 给 AI 立规矩：CLAUDE.md 与技能

AI 对你独有的约束一无所知——除非你告诉它。第一册第 12 章立下这条原则，这一章讲怎么把"告诉它"变成制度，而不是每次开工重复一遍。

制度的载体是一个放在仓库根目录的文件：CLAUDE.md——项目的宪法，Claude 每次在这个项目里开工都会先读它。写什么？四节足矣：这是什么（项目一句话简介、技术栈四大件）；怎么跑（常用命令：怎么装依赖、怎么本地启动）；规矩（代码风格、目录约定、"改动前先给计划"之类的流程要求）；红线（哪些文件绝对不许碰、哪些操作必须人来确认——我们把链上合约相关的一切列在这里，就是那次"链上考古"的制度化答案）。

比宪法更进一步的是技能（Skill）：可复用的专项规程。宪法管"在这个项目里的通则"，技能管"做某类事时的专门章程"——我们已经在真用了：硬件那套接线与调试规程是一个技能，工程护栏是另一个。通则写进 CLAUDE.md，专门章程拆成技能，AI 遇到对应场景自动取用。

规矩从哪来？从事故来。每次 AI 犯错，多问一句："哪条规矩能防止它再犯？"把答案写进宪法或技能——这套文件就成了团队的免疫系统：同一个坑，掉进去第二次才是真的亏。但反过来也有一条：规矩别贪多。宪法写成十页，每一条的分量都被稀释，AI 和人一样会"看不过来"。定期删掉过时的，只留真正重要的——精简本身就是维护。

### 关键领悟

不写规矩，你就得在每次对话里当复读机；写了规矩，你的经验教训才开始复利。

### 试一试

挑一个还没有 CLAUDE.md 的项目，写下第一版，就三节：这是什么、怎么跑、别碰什么。下次让 AI 干活，观察它有没有守规矩。

## 第 12 章 综合演练：从空文件夹到上线网址

本册所有技能，一次串起来。这套十步，也是你未来一切新项目的骨架。

1. 新建文件夹 `hello-web`，用 VS Code 打开它。
2. 让 Claude 插件生成一个最小的个人主页——就一个 `index.html`，越简单越好（这个文件本身，第三册第 2 章正好拿来解剖）。
3. 在源代码管理面板点"初始化仓库"（它背后就是 `git init`——给这个文件夹装上时间机器）。
4. `add`、`commit`，写下第一条存档说明："最小主页"。
5. GitHub 网页建一个空仓库 `hello-web`（复习：不勾任何初始化选项）。
6. 念三行咒语：`remote add origin`、`branch -M main`、`push -u origin main`。刷新网页，确认代码上云。
7. 打开 Vercel，Add New Project，Import 这个仓库，一路默认，Deploy。
8. 一分钟后，拿到一个 `xxx.vercel.app` 网址——发给爸爸，让他在手机上打开。这是你从零上线的第一个网站。
9. 回到 VS Code，把主页标题改一个词，`add`、`commit`、`push`。切到 Vercel 看着它自动重新构建、部署——第一册第 11 章画的那条流水线，此刻在你眼前真实转动。
10. 彩蛋（可选，做完等于提前修了第四册第一课）：去 Cloudflare，给我们的域名加一条 CNAME 记录指向这个 Vercel 项目——让它拥有一个真正属于我们的网址。

走完这十步，回头看：VS Code（1、2、9）、Git 四步循环（3、4、9）、GitHub 建仓上传（5、6）、部署流水线（7、8、9）、DNS（10）——本册和第一册在这里合龙。

### 关键领悟

从空文件夹到全世界可访问，十步、半小时。这件事在十年前需要一个团队。门槛已经消失了，剩下的只是把这十步走熟。

## 附：本册新词

- 活动栏 / 命令面板 / 集成终端：VS Code 的侧边导航 / 万能命令入口 (Cmd/Ctrl+Shift+P) / 内嵌的终端。
- 差异 (Diff)：两版文件的对比视图，红删绿增。
- 沙箱 (Sandbox)：隔离的云端工作环境，Claude Code 网页版干活的地方。
- 工作区 / 暂存区 / 仓库：稿纸 / 打包台 (add 的去处) / 存档柜 (commit 的去处)。
- 远程仓库 (Remote) / origin：云上的那份仓库 / 给它起的惯例联系人名。
- main：主分支的现行惯例名，"正式出版物"。
- Merge / 合并：把一条分支的成果并入另一条。
- 冲突 (Conflict) 与冲突标记：两边改同一处时 Git 的"你们自己挑"，`<<< === >>>` 三行便签。
- restore / revert / amend：撤销工作区改动 / 用反向存档抵消已推送的提交 / 改写最近一次未推送的提交。
- 改写历史：删改已共享的提交记录 (reset --hard、force push)，红线操作。
- scripts / npm run：package.json 里的快捷按钮区 / 按下快捷按钮。
- .env.example：环境变量清单模板，只列名字不含钥匙，可进 Git。
- CLAUDE.md / 技能 (Skill)：项目宪法 / 可复用的专项规程。
- 审查 (Review)：合并前正眼看完全部差异并给出意见的过程——本册真正想教会你的事。